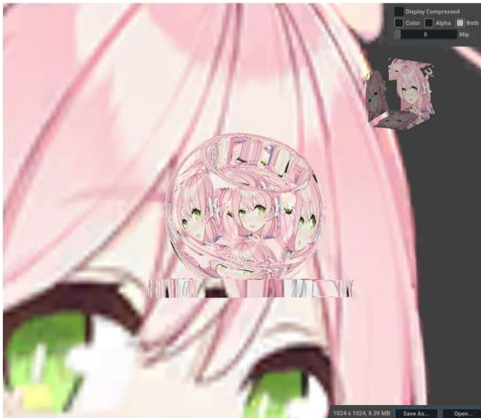




03 Texture

스카이 박스

게임 프로그래머 | 이창진



| 목차

Chapter 1. 스카이박스 정의

Chapter 2. 큐브매핑

Chapter 3. DDS 포맷

Chapter 4. 셰이더 코드

Chapter 5. 스카이박스 그리기

스카이박스란?

정의

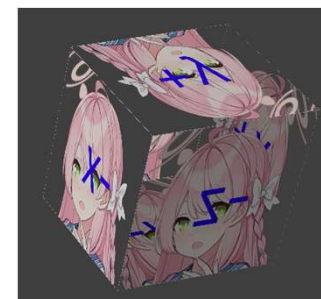
스카이박스(Skybox)는 3D 그래픽스 환경에서 장면의 배경을 표현하는 데 사용되는 환경 매핑 기술입니다.

이는 게임이나 시뮬레이션에서 플레이어가 거대한 공간 안에 있는 듯한 착각을 주도록 설계된 가상의 육면체 또는 구체입니다.

작동 방식

내부적으로 모든 오브젝트가 그려진 뒤에 렌더링됩니다

일반적으로 스카이박스는 다른 오브젝트에게 가려지는 경우가 많아 먼저 그리게 되면 두 번 그리는 문제가 생깁니다.



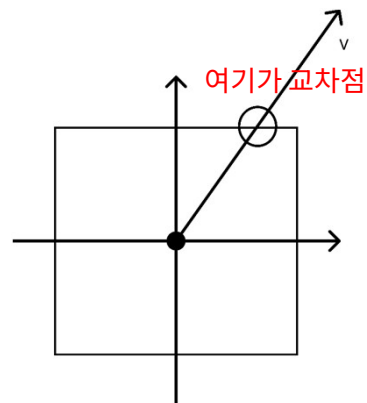
정의

정육면체는 6개의 면을 가집니다. 스카이 박스는 이 6개의 면에 6개의 텍스처 맵이 매핑됩니다.

정육면체의 중심을 원점으로하는 좌표축을 정렬해서 시스템을 만듭니다.

카메라에서 Eye, At, Up으로 만들어내는 Projection 행렬을 만드는 것을 떠올려보면 됩니다.

$+x$, $+y$, $+z$ 로 원점에서 방향벡터를 만들어 광선을 발사해 면과 교차한 교차점에 해당하는 색상을 얻을 수 있습니다.



d3d11.h 파일 내부

```
typedef
enum D3D11_TEXTURECUBE_FACE
{
    D3D11_TEXTURECUBE_FACE_POSITIVE_X= 0,
    D3D11_TEXTURECUBE_FACE_NEGATIVE_X= 1,
    D3D11_TEXTURECUBE_FACE_POSITIVE_Y= 2,
    D3D11_TEXTURECUBE_FACE_NEGATIVE_Y= 3,
    D3D11_TEXTURECUBE_FACE_POSITIVE_Z= 4,
    D3D11_TEXTURECUBE_FACE_NEGATIVE_Z= 5
} D3D11_TEXTURECUBE_FACE;
```

큐브매핑 - 비균등 스케일 문제



설명

법선벡터는 비균등 스케일에서 값이 달라집니다.

접선 벡터 $t = (0, 1, 0)$ 이라고
 법선 벡터 $n = (0, 0, 1)$ 이라 해봅시다
 그리고 변환 행렬은 다음과 같다고 해봅시다

$$M = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 3 & 1 \\ 0 & 1 & 1 \end{bmatrix}$$

변환 후 벡터들은 다음과 같습니다

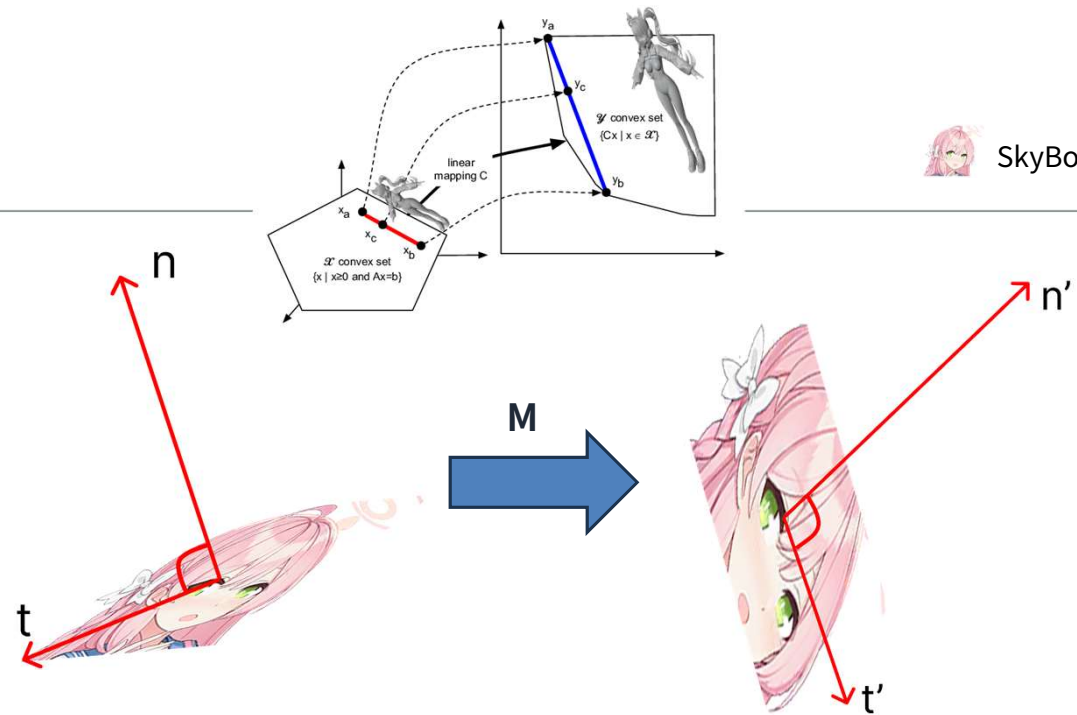
$$t' = Mt = (0, 3, 1)$$

$$n' = Mn = (0, 1, 1)$$

이 둘의 내적은 다음과 같습니다

$$t' \cdot n' = 0 \times 0 + 3 \times 1 + 1 \times 1 = 4$$

즉 수직성이 깨집니다.



해결하는 방법

법선 벡터 n 은 접선 벡터 t 과 수직하므로 항상

$$(n^T)t = 0$$

변환 후 법선 벡터 n' 과 접선 벡터 $t' = Mt$ 도 수직이어야 하므로,

$$(n')^T t' = (n')^T (Mt) = (n'^T M)t = 0$$

이 모양을 잘 보면

$$n^T = n'^T M$$

양변에 전치를 하고 정리하면

$$n' = (M^T)^{-1} n = ((M^{-1})^T) n$$

이 됩니다.

```
auto invWorlNormal = XMMatrixInverse(nullptr, m_baseProjection.world);
m_ConstantBuffer.worldInvTranspose = XMMatrixTranspose(invWorlNormal);
```



DDS는 GPU가 쓰는 압축된 텍스처입니다. 큐브맵, mip맵 체인, LOD 등을 전부 가지고 있습니다. 스카이박스, 알베도 맵, 반복 타일 등에서 효과적입니다.

GPU는 텍스처를 블록 단위, 캐시라인 단위로 읽습니다. 가까이 있는 물체를 원본 텍스처에서 샘플하면 이웃 픽셀들이 멀리 떨어진 주소를 자주 읽게되고 캐시가 자주 비게 됩니다. 반대로 숫자가 높은 LOD에서는 이웃 픽셀들이 가까운 주소에 있으니 캐시 블록을 재사용하게 됩니다. 캐시 히트레이트가 올라가게 되고 메모리 대역폭이 줄어들어 성능이 좋아집니다.

LOD		Resolution
LOD 0	->	256*256
LOD 1	->	128*128
LOD 2	->	32*32



여기에 접근할때보다

여기에 접근할 때가 주변이 더 가깝다



SkyBox.hlsl

```
TextureCube g_TexCube : register(t0);
SamplerState g_Sam : register(s0);
```

```
cbuffer CBChangesEveryFrame :
register(b0)
{
    matrix g_WorldViewProj;
}
```

```
struct SkyBoxVertexPos
{
    float3 posL : POSITION;
};
```

```
struct SkyBoxVertexPosHL
{
    float4 posH : SV_POSITION;
    float3 posL : POSITION;
};
```

SkyBoxPS

```
#include "10_Skybox.hlsl"
```

```
float4 PS(SkyBoxVertexPosHL pln) : SV_Target
{
    return g_TexCube.Sample(g_Sam,
        float3(pln.posL.x, pln.posL.y, -pln.posL.z));
}
```



텍스처 큐브의 부딪친 그 지점의 컬러값을 가져오기 위해 샘플을 반환합니다.

SkyBoxVS

```
#include "10_SkyBox.hlsl"
```

```
SkyBoxVertexPosHL VS(SkyBoxVertexPos vIn)
{
    SkyBoxVertexPosHL vOut;
```

// z/w = 1.0f 로 고정하여 원근 투영 변환의 영향을 받지 않도록 합니다.

// 왜 이렇게 하나면 스카이 박스는 아주 먼 플랜에 있다고 가정하기 때문입니다

```
float4 posH = mul(float4(vIn.posL, 1.0f), g_WorldViewProj);
vOut.posH = posH.xyww;
vOut.posL = vIn.posL;
return vOut;
}
```

SkyBox.hlsl

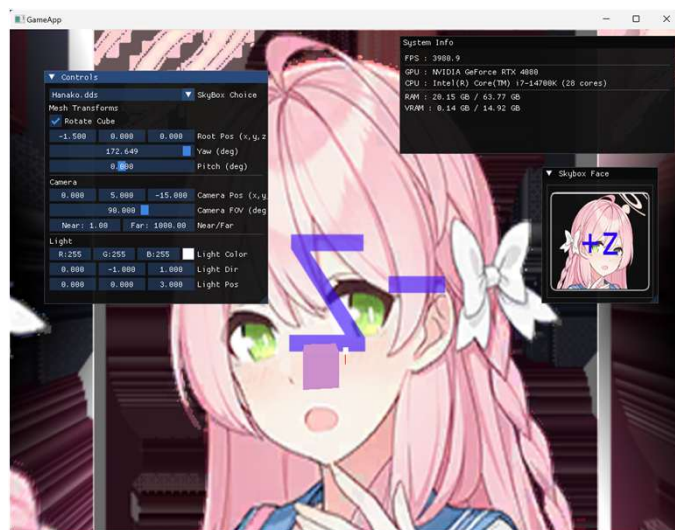
```
TextureCube g_TexCube : register(t0);
SamplerState g_Sam : register(s0);
```

```
cbuffer CBChangesEveryFrame :
register(b0)
{
    matrix g_WorldViewProj;
}
```

```
struct SkyBoxVertexPos
{
    float3 posL : POSITION;
};
```

```
struct SkyBoxVertexPosHL
{
    float4 posH : SV_POSITION;
    float3 posL : POSITION;
};
```

SkyBoxPS



텍스처 큐브의 부딪친 그 지점의 컬러값을 가져오기 위해 샘플을 반환합니다.

SkyBoxVS

```
#include "10_SkyBox.hlsl"
```

```
SkyBoxVertexPosHL VS(SkyBoxVertexPos vIn)
{
    SkyBoxVertexPosHL vOut;
```

// z/w = 1.0f 로 고정하여 원근 투영 변환의 영향을 받지 않도록 합니다.

// 왜 이렇게 하나면 스카이 박스는 아주 먼 플랜에 있다고 가정하기 때문입니다

```
float4 posH = mul(float4(vIn.posL, 1.0f), g_WorldViewProj);
vOut.posH = posH.xyww;
vOut.posL = vIn.posL;
return vOut;
}
```




SkyBox.hlsl

```
TextureCube g_TexCube : register(t0);
SamplerState g_Sam : register(s0);

cbuffer CBChangesEveryFrame :
register(b0)
{
    matrix g_WorldViewProj;
}

struct SkyBoxVertexPos
{
    float3 posL : POSITION;
};

struct SkyBoxVertexPosHL
{
    float4 posH : SV_POSITION;
    float3 posL : POSITION;
};
```

SkyBoxPS

```
#include "10_Skybox.hlsl"

float4 PS(SkyBoxVertexPosHL pIn) : SV_Target
{
    return g_TexCube.Sample(g_Sam, pIn.posL);
}
```



텍스처 큐브의 부딪친 그 지점의 컬러값을 가져오기 위해 샘플을 반환합니다.

SkyBoxVS

```
#include "10_SkyBox.hlsl"

SkyBoxVertexPosHL VS(SkyBoxVertexPos vIn)
{
    SkyBoxVertexPosHL vOut;

    // z/w = 1.0f 로 고정하여 원근 투영 변환의 영향을 받지 않도록
    // 합니다.
    // 왜 이렇게 하나면 스카이 박스는 아주 먼 플랜에 있다고 가정하기
    // 때문입니다
    float4 posH = mul(float4(vIn.posL, 1.0f), g_WorldViewProj);
    vOut.posH = posH.xyww;
    vOut.posL = vIn.posL;
    return vOut;
}
```

SkyBox.hlsl

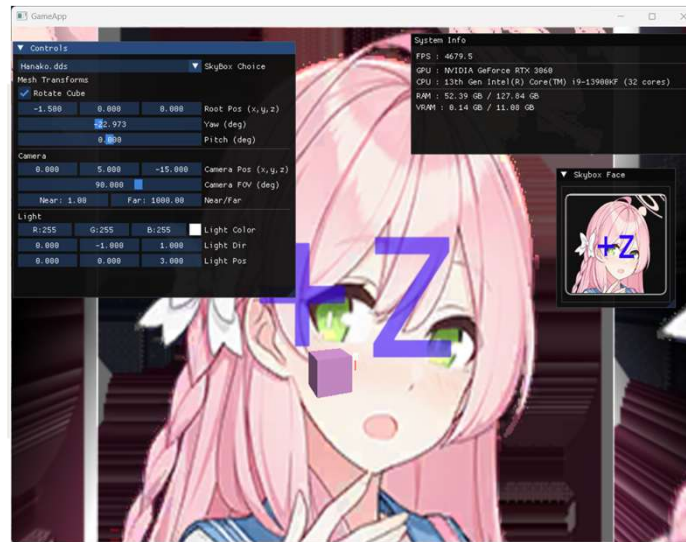
```
TextureCube g_TexCube : register(t0);
SamplerState g_Sam : register(s0);
```

```
cbuffer CBChangesEveryFrame :
register(b0)
{
    matrix g_WorldViewProj;
}
```

```
struct SkyBoxVertexPos
{
    float3 posL : POSITION;
};
```

```
struct SkyBoxVertexPosHL
{
    float4 posH : SV_POSITION;
    float3 posL : POSITION;
};
```

SkyBoxPS



텍스처 큐브의 부딪친 그 지점의 컬러값을 가져오기 위해 샘플을 반환합니다.

SkyBoxVS

```
#include "10_SkyBox.hlsl"
```

```
SkyBoxVertexPosHL VS(SkyBoxVertexPos vIn)
{
    SkyBoxVertexPosHL vOut;
```

// z/w = 1.0f 로 고정하여 원근 투영 변환의 영향을 받지 않도록 합니다.

// 왜 이렇게 하나면 스카이 박스는 아주 먼 플랜에 있다고 가정하기 때문입니다

```
float4 posH = mul(float4(vIn.posL, 1.0f), g_WorldViewProj);
vOut.posH = posH.xyww;
vOut.posL = vIn.posL;
return vOut;
}
```

스카이박스 그리기

코드

```
ID3D11DepthStencilState* prevDS = nullptr;
UINT prevRef = 0;
m_pDeviceContext->OMGetDepthStencilState(&prevDS, &prevRef);
```

```
// 스텐실 비활성 상태(한 번만 생성해 재사용)
static ID3D11DepthStencilState* dsNoStencil = nullptr;
if (!dsNoStencil) {
    D3D11_DEPTH_STENCIL_DESC dsd = {};
    dsd.DepthEnable = TRUE;
    dsd.DepthWriteMask = D3D11_DEPTH_WRITE_MASK_ALL; //
    깊이는 필요에 맞게
    dsd.DepthFunc = D3D11_COMPARISON_LESS_EQUAL;
    dsd.StencilEnable = FALSE; // 스텐실 끄
    m_pDevice->CreateDepthStencilState(&dsd, &dsNoStencil);
}
```

```
// 적용 후 원하는 드로우 작업
m_pDeviceContext->OMSetDepthStencilState(dsNoStencil, 0);
// ... draw ...
```

```
// 원상복구
m_pDeviceContext->OMSetDepthStencilState(prevDS, prevRef);
SAFE_RELEASE(prevDS);
```

설명

큰 박스의 안에 있으니 당연히
CCW를 반대로 만들어주고 **깊이**
스텐실도 잠시 꺼두고 그려야 합니다.

스텐실을 끄고 다시 복귀하는
코드입니다.



스카이박스 그리기2

시도

매우 큰 박스가 있고 이걸 카메라가 안에서 본다고 생각을 하고 시도해 봤습니다.

큰 박스의 안에 있으니 당연히 CCW를 반대로 만들어주고 깊이 스텐실도 잠시 꺼두고 그려야 합니다.

안에서 밖을 보는 것이니 단순히 카메라의 projection 행렬에 음수를 곱해 넣으면 되지 않을까 생각했지만. 이렇게 그리게 되면 z 축으로 반전되어 나타납니다

- P: 투영행렬, V: 뷰행렬, R,t는 뷰의 회전/이동
- $V0 = [R^T \ 0; 0 \ 1]$ (카메라 평행이동 제거 뷰)
- ConstantBuffer에는 전치 행렬을 넣으므로 모두 T 형태

$$\bullet wvpT = (-P \cdot V0)^T = -P^T \cdot V0^T$$

코드

```
XMMATRIX viewT = m_baseProjection.view; // transposed view
XMMATRIX view = XMMatrixTranspose(viewT);
view.r[3] = XMVectorSet(0.0f, 0.0f, 0.0f, XMVectorGetW(view.r[3]));
XMMATRIX viewNoTransT = XMMatrixTranspose(view);
XMMATRIX wvpT = XMMatrixMultiply(-m_baseProjection.proj,
viewNoTransT);
```



스카이박스 그리기3



해결

생각을 해보니 카메라가 박스의 안에서 박스의 안쪽면을 보고 있는데 텍스처는 큐브의 밖에 매핑이 되어 있습니다 즉 좌표계 플립은 따로 z축 스케일을 -1만큼 해서 넣어주면 됩니다 -> 라고 생각했지만, 이것도 올바른 해결방법이 아니었습니다. 이렇게 하면 셰이더에서 다시 z스케일에 -1을 곱해야합니다.

View 행렬에 이동 행렬만 제거해버리고 회전만 반영되게 했습니다.

- P: 투영행렬, V: 뷰행렬, R,t는 뷰의 회전/이동
- $V0 = [R^T \ 0; 0 \ 1]$ (카메라 평행이동 제거 뷰)
- $Fz = \text{diag}(1, 1, -1)$ (Z축 플립; 큐브 안쪽을 LH 규약과 맞춤)
- ConstantBuffer에는 전치 행렬을 넣으므로 모두 T 형태

$$g_WorldViewProj = (P \cdot V0 \cdot Fz)^T$$

코드

```
XMMATRIX viewT = m_baseProjection.view; // transposed view
XMMATRIX view = XMMatrixTranspose(viewT);
view.r[3] = XMVectorSet(0.0f, 0.0f, 0.0f, XMVectorGetW(view.r[3]));
XMMATRIX viewNoTransT = XMMatrixTranspose(view);
XMMATRIX wvpT = XMMatrixMultiply(m_baseProjection.proj, viewNoTransT);
```

```
D3D11_RASTERIZER_DESC rd{};
rd.CullMode = D3D11_CULL_NONE;
```



팁 - dds 텍스처 뷰어

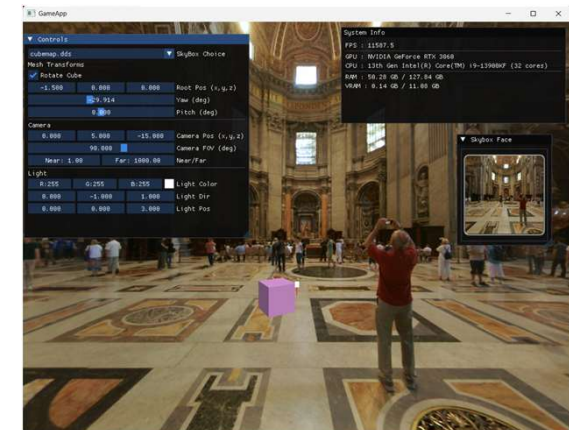
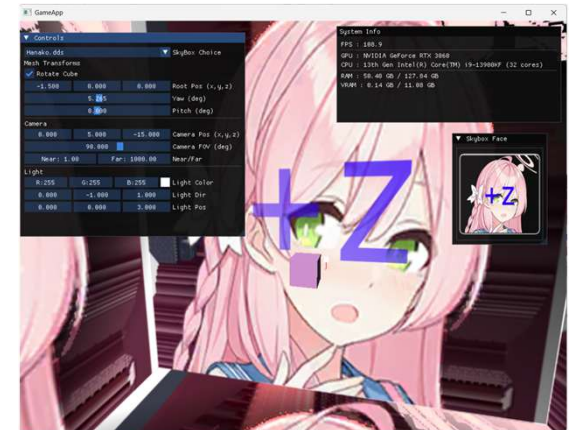


```
void App::PrepareSkyFaceSRVs()
{
    Microsoft::WRL::ComPtr<ID3D11Resource> res;
    m_pTextureSRV->GetResource(res.GetAddressOf());
    if (!res) return;

    Microsoft::WRL::ComPtr<ID3D11Texture2D> tex2D;
    HR_T(res.As(&tex2D));
    D3D11_TEXTURE2D_DESC desc{};

    tex2D->GetDesc(&desc);
    if ((desc.ArraySize < 6)) return;

    for (UINT face = 0; face < 6; ++face)
    {
        D3D11_SHADER_RESOURCE_VIEW_DESC sd{};
        sd.Format = desc.Format;
        ... // sd 값 넣어주기
        ID3D11ShaderResourceView* faceSRV = nullptr;
        if (SUCCEEDED(m_pDevice->CreateShaderResourceView(tex2D.Get(), &sd, &faceSRV)))
        {
            m_pSkyFaceSRV[face] = faceSRV;
        }
    }
}
```



감사합니다.